Project Title: Credit Score Classification

- **Team Members:**
- Berke DURDU (2309111017)
- Gabriel KESLER (2309111041)
- Kerem KOÇAK (2309111056)

- **Course:** COE305 MACHINE LEARNING
- **Date:** 03.01.2026

Problem Statement & Motivation

What problem are you solving?

A Multi-Class Classification task to predict the Credit_Score of individuals (Standard, Poor, Good) based on their financial history.

Why is it important?

Financial institutions need accurate risk assessment to minimize default rates.

Manual classification is inefficient; an automated model provides instant and consistent credit scoring.

Why Machine Learning?

The dataset contains 73,709 samples with complex, non-linear relationships between income, debt, and payment behavior.

Rule-based systems cannot effectively handle high-dimensional data or capture subtle patterns like "Credit Mix" interactions.

Dataset Overview

Dataset Source: Kaggle - Credit Score Classification.

Type: Structured / Tabular Data.

No. of Samples: 73,709 (after cleaning).

No. of Features:

25 Features: In the saved cleaned_data.csv (raw cleaned data).

49 Features: During model training. *Note: One-Hot Encoding is applied within the training Pipeline, expanding the feature space dynamically*.

Target Variable: Credit_Score (Classes: Standard, Poor, Good).

Data Preprocessing & EDA

Missing Value Handling:

- Numerical values imputed with **Median**; Categorical values imputed with **Mode**.

Data Cleaning:

- Used **Regex** to fix typos in 'Age' and 'Annual_Income' (e.g., cleaning "24_" to 24), recovering thousands of rows.

```python
# 2. Data Cleaning
df_cleaned = df.copy()
cols_drop = [c for c in ['ID', 'SSN', 'Name'] if c in df_cleaned.columns]
df_cleaned = df_cleaned.drop(columns=cols_drop)
df_cleaned = df_cleaned.drop_duplicates()
cols_to_fix = ['Age', 'Annual_Income', 'Outstanding_Debt', 'Monthly_Inhand_Salary']
for col in cols_to_fix:
    if col in df_cleaned.columns:
        df_cleaned[col] = df_cleaned[col].astype(str).str.replace('_', '', regex=False)
        df_cleaned[col] = df_cleaned[col].astype(str).str.replace(r'[^\d.-]', '', regex=True)
        df_cleaned[col] = pd.to_numeric(df_cleaned[col], errors='coerce')

df_cleaned.loc[df_cleaned['Age'] > 100, 'Age'] = np.nan
df_cleaned.loc[df_cleaned['Monthly_Inhand_Salary'] < 0, 'Monthly_Inhand_Salary'] = np.nan
```

```python
# 3. Missing Value Handling
numeric_cols = df_cleaned.select_dtypes(include=[np.number]).columns
for col in numeric_cols:
    df_cleaned[col].fillna(df_cleaned[col].median(), inplace=True)

categorical_cols = df_cleaned.select_dtypes(include=['object']).columns
for col in categorical_cols:
    mode_val = df_cleaned[col].mode()[0] if not df_cleaned[col].mode().empty else 'Unknown'
    df_cleaned[col].fillna(mode_val, inplace=True)
```

Data Preprocessing & EDA

- **Outlier Handling:** Applied IQR method to clip extreme values in financial columns.
- **Encoding Strategy:**
- **Label Encoding:** Used for ordinal data like 'Credit_Mix' (Bad: -1, Standard: 0, Good: 1).
- **One-Hot Encoding:** Applied via **Pipeline** for nominal features (e.g., Occupation), ensuring transformations occur during training without data leakage.
- **Key EDA Insight:**
- **Class Imbalance:** "Standard" is the majority class (~53.2%), while "Good" is the minority (~17.8%).
- **Income Distribution:** Annual income is right-skewed.

```python
# 5. Encoding
# Label Encoding
df_cleaned = df_cleaned[df_cleaned['Credit_Mix'] != '_']
df_cleaned = df_cleaned[df_cleaned['Payment_Behaviour'] != '!@9#%8']

if 'Payment_of_Min_Amount' in df_cleaned.columns:
    df_cleaned['Payment_of_Min_Amount'] = df_cleaned['Payment_of_Min_Amount'].map({'No': 0, 'Yes': 1})
if 'Credit_Mix' in df_cleaned.columns:
    df_cleaned['Credit_Mix'] = df_cleaned['Credit_Mix'].map({'Bad': -1, 'Standard': 0, 'Good': 1})

df_analysis = df_cleaned.copy()

# One-Hot Encoding
df_cleaned = pd.get_dummies(df_cleaned, columns=['Occupation', 'Payment_Behaviour', 'Month'], drop_first=True)
```

```python
# 4. Outlier Handling
numeric_features = df_cleaned.select_dtypes(include=[np.number]).columns
for col in numeric_features:
    Q1 = df_cleaned[col].quantile(0.25)
    Q3 = df_cleaned[col].quantile(0.75)
    IQR = Q3 - Q1
    df_cleaned[col] = df_cleaned[col].clip(lower=Q1 - 1.5*IQR, upper=Q3 + 1.5*IQR)
```

```
Train: (58967, 24), Test: (14742, 24)
Target Classes: ['Good' 'Poor' 'Standard']
Encoded as: [0, 1, 2]
Class distribution:
2    0.532450
1    0.289806
0    0.177743
```

Ensemble Learning Models

Models Selected:

Gradient Boosting

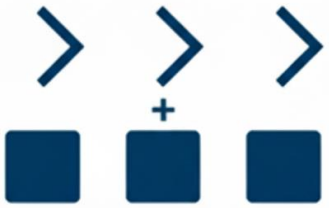
The Iterative Improver.

A powerful ensemble method selected for its technique of sequentially correcting the errors of previous models, often leading to high accuracy.

Random Forest

The Non-Linear Powerhouse.

Chosen for its proven ability to capture complex interactions between features without requiring extensive tuning.



XGBoost

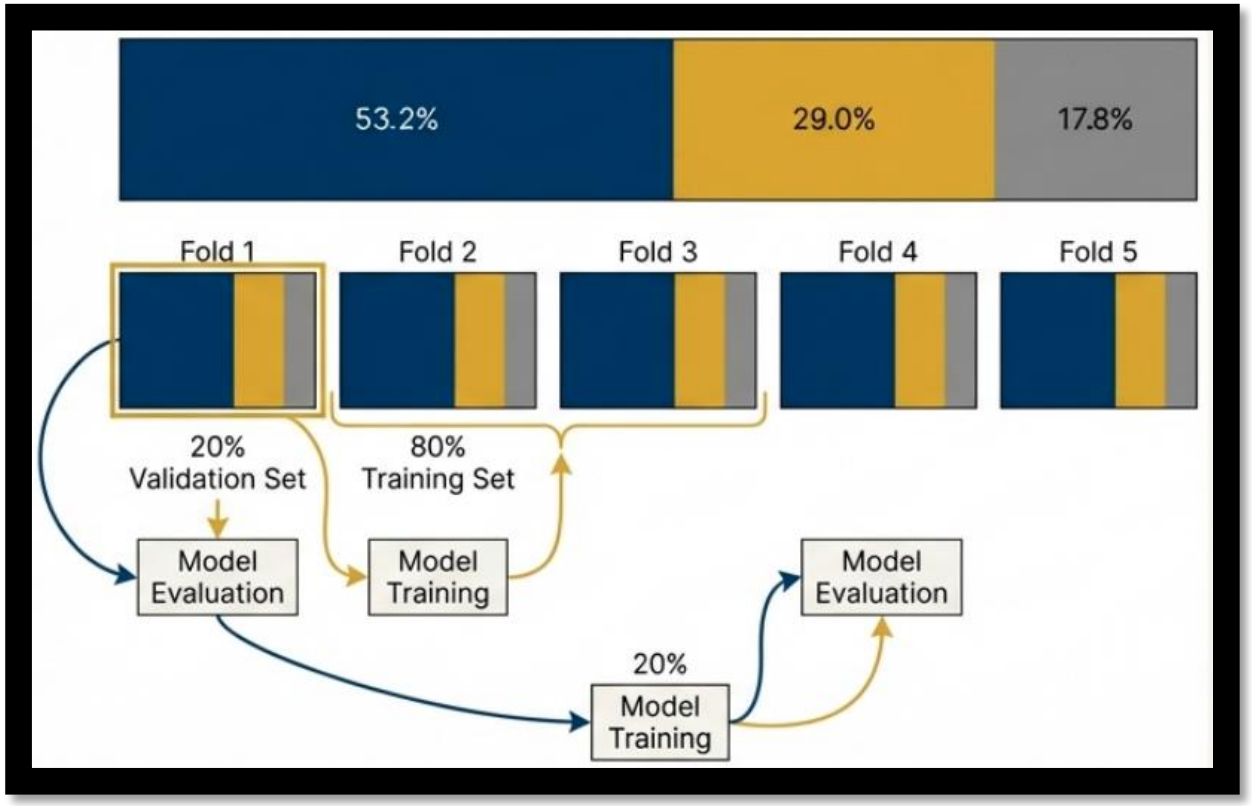
The Optimized Boosting Machine.

A high-performance, scalable ensemble method utilizing advanced regularation for speed and accuracy.

Ensemble Learning Models

Cross-Validation Setup:

- We used a Stratified 5-Fold approach to ensure that the class imbalance was preserved across all folds, leading to a reliable performance estimate.



Baseline Results (Test Data)

Metrics Used: Accuracy, Precision, Recall, F1-score.

Model	Accuracy	Precision	Recall	F1-Score
Random Forest	**0.7877**	**0.7886**	**0.7877**	**0.7881**
XGBoost	0.7397	0.7414	0.7397	0.7402
Gradient Boosting	0.7129	0.7182	0.7129	0.7141

Analysis: Random Forest significantly outperforms both boosting methods across all key metrics. Its default parameters appear well-suited to the dataset's structure.

Hyperparameter Tuning

Hyperparameter Tuning

Tuning Strategy

- Method Used: RandomizedSearchCV

- Cross-validation Folds: 3

- Scoring Metric: F1-Score (weighted)

- Number of Iterations: 5

RandomizedSearchCV: Selected over GridSearchCV for computational efficiency.

Approach: Samples a fixed number of parameter combinations instead of an exhaustive search.

Benefit: Explores a wider hyperparameter space faster, finding near-optimal values within reasonable time constraints.

Final Results & Comparison

Ensemble Learning Models Before and After Hyperparameter Tuning

Ensemble Models Performance (Before Tuning)

Model	Accuracy	Precision	Recall	F1-Score
Random Forest	0.7877	0.7886	0.7877	0.7881
Gradient Boosting	0.7129	0.7182	0.7129	0.7141
XGBoost	0.7397	0.7414	0.7397	0.7402

Ensemble Models Performance (After Tuning)

Model	Accuracy	Precision	Recall	F1-Score
Random Forest	0.7889	0.7898	0.7889	0.7892
Gradient Boosting	0.7538	0.7545	0.7538	0.754
XGBoost	0.7724	0.7729	0.7724	0.7726

Conclusion & Best Model

- **Top Performer: Random Forest** secured the highest **Accuracy (78.89%)** and **F1-Score (0.7892)**.
- **Key Insight:** The untuned **Random Forest** model still performed better than the fully tuned versions of **XGBoost** and **Gradient Boosting**.
- **Final Verdict:** Due to its unmatched consistency and high scores, **Random Forest** is the chosen model.

Ensemble Models Performance Comparison (F1-Score Ranked)

Model	Tuning Status	Accuracy	Precision	Recall	F1-Score
Random Forest	After Tuning	0.7889	0.7898	0.7889	0.7892
Random Forest	Before Tuning	0.7877	0.7886	0.7877	0.7881
XGBoost	After Tuning	0.7724	0.7729	0.7724	0.7726
XGBoost	Before Tuning	0.7397	0.7414	0.7397	0.7402
Gradient Boosting	After Tuning	0.7538	0.7545	0.7538	0.7540
Gradient Boosting	Before Tuning	0.7129	0.7182	0.7129	0.7141

Final Recommendation

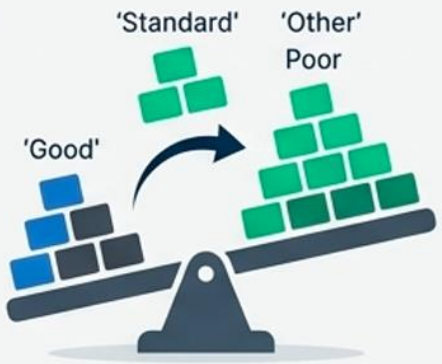
After rigorous evaluation, the **Tuned Random Forest Classifier** is identified as the optimal model for this credit score classification task, surpassing all other ensemble methods.

Justification by Evidence

Superior Performance: Achieved the highest **Accuracy (0.7889)** and **F1-Score (0.7892)** on the test set, demonstrating peak predictive power.

Robustness: The untuned Random Forest model outperformed the tuned versions of both XGBoost and Gradient Boosting.

The Investigation Continues: A Roadmap for Future Improvements



1. Handle Class Imbalance

Implement advanced sampling techniques like **SMOTE** (Synthetic Minority Over-sampling Technique) to re-balance the dataset and potentially improve recall for the 'Good' credit score class.

2. Explore Advanced Algorithms

Experiment with state-of-the-art boosting libraries such as **LightGBM** and **CatBoost**, which are known for their performance and efficiency on tabular data.

3. Refine Hyperparameter Tuning

Conduct a more exhaustive search for Random Forest's parameters using `GridSearchCV` or a more extensive `RandomizedSearchCV` to confirm if performance can be improved.

4. Implement Feature Selection

Use the feature importance scores from Random Forest to select the most impactful features, potentially creating a simpler, faster, and more interpretable model without sacrificing performance.